(a) Define primary clustering in context of hashing? Explain the effect of primary clustering and the mechanism to reduce it. [4]

(b) Consider inserting the keys $10, 22, 31, 4, 15, 28, 17, 88, 59$ into a hash table of length $m = 11$ using open addressing with the auxiliary hash function $h'(k) = k \ mod \ 11$. Illustrate the result of inserting these keys using i) quadratic probing with $C_1 = 1$ and $C_2 = 3$, and ii) using double hashing with $h_1(k) = k \ mod \ 11$ and $h_2(k) = 1 + (k \ mod \ (m-1))$. [6]

(c) Proof that the expected number of probes in an unsuccessful search is at most $1/(1-\alpha)$ for an open-address hash table with load factor $\alpha = n/m < 1$. Assuming uniform hashing. [4]

Definition

Primary clustering is the formation of long consecutive groups of occupied slots in a hash table when collisions are resolved using **linear probing**.

In linear probing,

$$h(k,i) = \big(h'(k) + i\big) \bmod m.$$

When a collision occurs, the algorithm checks the next slot, then the next one, and so on. As a result, consecutive occupied blocks called **clusters** are created.

For example:

3	8	15	22	

The four consecutive occupied positions form a cluster.

Effect of primary clustering

Primary clustering causes the following problems:

1. A key hashing anywhere inside or immediately before the cluster is inserted at the end of the cluster.
2. Therefore, the cluster becomes larger.
3. As the cluster grows, more keys collide with it.
4. Successful search, unsuccessful search, and insertion require more probes.
5. Two nearby clusters may join together and form one larger cluster.

Thus, primary clustering causes the performance of linear probing to deteriorate significantly, especially when the load factor becomes high. Levitin also notes that larger clusters are more likely to attract new keys and merge with other clusters. 212803334-Intro-to-Design-Analy...

Mechanisms for reducing primary clustering

Primary clustering can be reduced by using:

- **Quadratic probing**, where the probe distance grows quadratically:

$$h(k,i) = \big(h'(k) + c_1 i + c_2 i^2\big) \bmod m.$$

- **Double hashing**, where the step size depends on a second hash function:

$$h(k,i) = \big(h_1(k) + i\, h_2(k)\big) \bmod m.$$

- Keeping the load factor sufficiently low.
- Rehashing all keys into a larger table when the table becomes too full.

Double hashing is generally more effective because different keys can have different initial positions as well as different step sizes.

2. Insert the keys using quadratic probing and double hashing

Given:

$$m = 11$$

Keys:

$$10, 22, 31, 4, 15, 28, 17, 88, 59$$

Primary hash function:

$$h'(k) = k \bmod 11.$$

The table positions are:

$$0, 1, 2, \ldots, 10.$$

(i) Quadratic Probing

Given:

$$c_1 = 1, \qquad c_2 = 3.$$

Therefore,

$$\boxed{h(k, i) = \left(k \bmod 11 + i + 3i^2\right) \bmod 11}$$

for

$$i = 0, 1, 2, \ldots$$

We always begin with $i = 0$.

Insert 10

$$h(10, 0) = 10 \bmod 11 = 10.$$

Slot 10 is empty.

$$10 \to T[10]$$

Insert 22

$$h(22, 0) = 22 \bmod 11 = 0.$$

Slot 0 is empty.

$$22 \to T[0]$$

Insert 31

$$h(31, 0) = 31 \bmod 11 = 9.$$

Slot 9 is empty.

$$31 \to T[9]$$

Insert 4

$$h(4, 0) = 4.$$

Slot 4 is empty.

$$4 \to T[4]$$

Insert 15

Initial position:

$$h(15, 0) = 15 \bmod 11 = 4.$$

Slot 4 is occupied by 4.

For $i = 1$,

$$\begin{aligned} h(15, 1) &= (4 + 1 + 3(1)^2) \bmod 11 \\ &= 8. \end{aligned}$$

Slot 8 is empty.

$$15 \to T[8]$$

Probe sequence:

$$4 \to 8.$$

Insert 28

$$h(28, 0) = 28 \bmod 11 = 6.$$

Slot 6 is empty.

$$28 \to T[6]$$

Insert 17

Initial position:

$$h(17, 0) = 17 \bmod 11 = 6.$$

Slot 6 is occupied.

For $i = 1$,

$$h(17, 1) = (6 + 1 + 3) \bmod 11 = 10.$$

Insert 88

Initial position:

$$h(88, 0) = 88 \bmod 11 = 0.$$

Now calculate successively:

i	$h(88, i)$	Condition
0	0	occupied
1	$(0 + 1 + 3) \bmod 11 = 4$	occupied
2	$(0 + 2 + 12) \bmod 11 = 3$	occupied
3	$(0 + 3 + 27) \bmod 11 = 8$	occupied
4	$(0 + 4 + 48) \bmod 11 = 8$	occupied
5	$(0 + 5 + 75) \bmod 11 = 3$	occupied
6	$(0 + 6 + 108) \bmod 11 = 4$	occupied
7	$(0 + 7 + 147) \bmod 11 = 0$	occupied
8	$(0 + 8 + 192) \bmod 11 = 2$	empty

Therefore,

$$88 \to T[2].$$

Probe sequence:

$$0 \to 4 \to 3 \to 8 \to 8 \to 3 \to 4 \to 0 \to 2.$$

Notice that quadratic probing may repeat some positions before reaching an empty position.

Insert 59

Share

Initial position:

$$h(59, 0) = 59 \bmod 11 = 4.$$

Slot 4 is occupied.

For $i = 1$,

$$h(59, 1) = (4 + 1 + 3) \bmod 11 = 8.$$

Slot 8 is occupied.

For $i = 2$,

$$\begin{aligned} h(59, 2) &= (4 + 2 + 12) \bmod 11 \\ &= 18 \bmod 11 \\ &= 7. \end{aligned}$$

Slot 7 is empty.

$$59 \to T[7].$$

Probe sequence:

$$4 \to 8 \to 7.$$

Final quadratic-probing table

Index	0	1	2	3	4	5	6	7	8	9	10
Key	22	—	88	17	4	—	28	59	15	31	10

(ii) Double Hashing

Given:

$$h_1(k) = k \bmod 11$$

and

$$h_2(k) = 1 + (k \bmod (m-1)).$$

Since $m = 11$,

$$\boxed{h_2(k) = 1 + (k \bmod 10)}.$$

The complete probe function is

$$\boxed{h(k,i) = (h_1(k) + i\,h_2(k)) \bmod 11}$$

Double hashing generates successive probes separated by the fixed key-dependent step $h_2(k)$.

Introduction_to_Algorithms_4th_...

Insert 10

$$h_1(10) = 10, \qquad h_2(10) = 1 + (10 \bmod 10) = 1.$$

For $i = 0$,

$$h(10,0) = 10.$$

Thus,

$$10 \to T[10].$$

Insert 22

$$h_1(22) = 0, \qquad h_2(22) = 1 + 2 = 3.$$

$$h(22, 0) = 0.$$

Thus,

$$22 \to T[0].$$

Insert 31

$$h_1(31) = 9, \qquad h_2(31) = 1 + 1 = 2.$$

$$h(31, 0) = 9.$$

Thus,

$$31 \to T[9].$$

Insert 4

$$h_1(4) = 4, \qquad h_2(4) = 1 + 4 = 5.$$

$$h(4, 0) = 4.$$

Thus,

$$4 \to T[4].$$

Insert 15

$$h_1(15) = 4, \qquad h_2(15) = 1 + 5 = 6.$$

For $i = 0$,

$$h(15, 0) = 4,$$

which is occupied.

For $i = 1$,

$$h(15, 1) = (4 + 6) \bmod 11 = 10,$$

which is occupied.

For $i = 2$,

$$h(15, 2) = (4 + 2 \cdot 6) \bmod 11 = 16 \bmod 11 = 5.$$

Slot 5 is empty.

$$15 \to T[5].$$

Probe sequence:

$$4 \to 10 \to 5.$$

Insert 28

$$h_1(28) = 6, \qquad h_2(28) = 1 + 8 = 9.$$

$$h(28, 0) = 6.$$

Thus,

$$28 \to T[6].$$

Insert 17

$$h_1(17) = 6, \qquad h_2(17) = 1 + 7 = 8.$$

For $i = 0$,

$$h(17, 0) = 6,$$

which is occupied.

For $i = 1$,

$$h(17, 1) = (6 + 8) \bmod 11 = 14 \bmod 11 = 3.$$

Slot 3 is empty.

$$17 \to T[3].$$

Insert 59

$$h_1(59) = 4, \qquad h_2(59) = 1 + 9 = 10.$$

For $i = 0$,

$$h(59, 0) = 4,$$

which is occupied.

For $i = 1$,

$$h(59, 1) = (4 + 10) \bmod 11 = 3,$$

which is occupied.

For $i = 2$,

$$h(59, 2) = (4 + 20) \bmod 11 = 24 \bmod 11 = 2.$$

Slot 2 is empty.

$$59 \to T[2].$$

Probe sequence:

$$4 \to 3 \to 2.$$

Final double-hashing table

Index	0	1	2	3	4	5	6	7	8	9	10
Key	22	—	59	17	4	15	28	88	—	31	10

Therefore,

$$\boxed{[22,\ -,\ 59,\ 17,\ 4,\ 15,\ 28,\ 88,\ -,\ 31,\ 10]}$$

3. Expected probes in an unsuccessful search

Theorem

For an open-address hash table with load factor

$$\alpha = \frac{n}{m} < 1,$$

the expected number of probes in an unsuccessful search is at most

$$\boxed{\frac{1}{1-\alpha}},$$

assuming independent uniform permutation hashing.

Proof

Let X be the number of probes made in an unsuccessful search.

For the search to require at least i probes, the first $i-1$ probed slots must all be occupied.

Therefore,

$$P(X \geq i) = \frac{n}{m} \cdot \frac{n-1}{m-1} \cdot \frac{n-2}{m-2} \cdots \frac{n-i+2}{m-i+2}.$$

Since

$$\frac{n-j}{m-j} \leq \frac{n}{m} = \alpha$$

for $0 \leq j < m$, we get

$$P(X \geq i) \leq \alpha^{i-1}.$$

Now use the tail-sum formula for a positive integer-valued random variable:

$$E[X] = \sum_{i=1}^{\infty} P(X \geq i).$$

Hence,

$$E[X] \leq \sum_{i=1}^{\infty} \alpha^{i-1}$$
$$= \sum_{j=0}^{\infty} \alpha^{j}.$$

Since $\alpha < 1$, this is a geometric series:

$$\sum_{j=0}^{\infty} \alpha^{j} = \frac{1}{1-\alpha}.$$

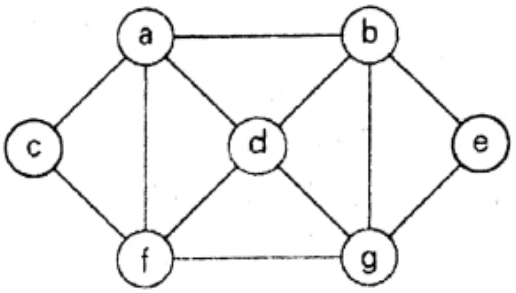
Therefore,

$$\boxed{E[X] \leq \frac{1}{1-\alpha}}.$$

Hence proved. This is the same argument used in CLRS Theorem 11.6. Introduction_to_Algorithms_4th_...

2 (a) Define the state-space tree. Describe the features of a node of state-space tree in context of Backtracking. **[3]**

(b) Design a linear-time algorithm that finds a solution to the n-queens problem for any $n \geq 4$. **[5]**



(c) Apply backtracking to the problem of finding a Hamiltonian circuit in the above graph. **[6]**

2(a) State-Space Tree in Backtracking

Definition

A **state-space tree** is a rooted tree representing all possible states or partial solutions of a problem.

- The **root** represents the initial state or empty solution.
- Each **edge** represents a choice.
- Each **node** represents a partial solution.
- A path from the root to a leaf represents a complete candidate solution.

A backtracking algorithm explores this tree depth-first and stops exploring a branch when its node becomes nonpromising. Levitin describes each node as a partially constructed tuple $(x_1, x_2, \ldots, x_i)$.

212803334-Intro-to-Design-Analy...

Features of a node

A node in a backtracking state-space tree may be:

1. **Promising node:** The partial solution satisfies all constraints and may lead to a complete solution. Therefore, its children are generated.
2. **Nonpromising node:** The partial solution violates a constraint or cannot lead to a solution. Therefore, the branch is pruned.
3. **Solution node:** The node represents a complete solution satisfying all constraints.
4. **Dead-end node:** No valid choice can be made from this node, so the algorithm backtracks to its parent.

Exam conclusion

> Thus, backtracking constructs a solution incrementally by exploring promising nodes and pruning nonpromising nodes in the state-space tree.

2(b) Linear-Time Algorithm for the n-Queens Problem

We represent a solution by

$$X = (x_1, x_2, \ldots, x_n),$$

where x_i is the column containing the queen in row i.

The following constructive method produces a solution for every

$$n \geq 4.$$

The book's exercise hint says to consider the six possible values of $n \bmod 6$; the cases $n \bmod 6 = 2$ and $n \bmod 6 = 3$ need special adjustments. 212803334-Intro-to-Design-Analy...

Algorithm

Case 1: $n \bmod 6 \neq 2$ **and** $n \bmod 6 \neq 3$

Place queens in the columns:

$$2, 4, 6, \ldots, n,\ 1, 3, 5, \ldots, n-1.$$

That is, write all even numbers first, followed by all odd numbers.

Case 2: $n \bmod 6 = 2$

Place queens in the order:

$$2, 4, 6, \ldots, n,\ 3, 1, 7, 9, 11, \ldots, n-1,\ 5.$$

Case 3: $n \bmod 6 = 3$

Place queens in the order:

$$4, 6, 8, \ldots, n-1,\ 2,\ 5, 7, 9, \ldots, n,\ 1, 3.$$

Each number is generated only once, so the running time is

$$\boxed{O(n)}.$$

The space required to store the arrangement is

$$\boxed{O(n)}.$$

Pseudocode

```
LINEAR-N-QUEENS(n)

if n mod 6 ≠ 2 and n mod 6 ≠ 3
    output 2, 4, 6, ..., largest even ≤ n
    output 1, 3, 5, ..., largest odd ≤ n

else if n mod 6 = 2
    output 2, 4, 6, ..., n
    output 3, 1
    output 7, 9, 11, ..., n - 1
    output 5

else
    output 4, 6, 8, ..., n - 1
    output 2
    output 5, 7, 9, ..., n
    output 1, 3
```

Example: $n = 8$

Since

$$8 \bmod 6 = 2,$$

the arrangement is

$$\boxed{X = (2, 4, 6, 8, 3, 1, 7, 5)}.$$

Thus:

Row	Column
1	2
2	4
3	6
4	8
5	3
6	1
7	7
8	5

No two queens are in the same row, column, or diagonal.

2(c) Hamiltonian Circuit Using Backtracking

Given graph

From the diagram, the adjacency lists are:

$$
\begin{aligned}
a &: \ b, c, d, f \\
b &: \ a, d, e, g \\
c &: \ a, f \\
d &: \ a, b, f, g \\
e &: \ b, g \\
f &: \ a, c, d, g \\
g &: \ b, d, e, f
\end{aligned}
$$

We fix a as the starting vertex to avoid generating rotationally equivalent circuits.

A vertex is promising when:

- It is adjacent to the previously selected vertex.
- It has not already appeared in the path.
- At the final step, it is adjacent to the starting vertex a.

Backtracking process

Starting from a, choose adjacent vertices alphabetically.

First attempt

$$a \to b \to d \to f \to c$$

From c, the only adjacent vertices are a and f, both already used. However, vertices e and g remain unvisited.

Therefore, this is a dead end:

$$a - b - d - f - c \quad \text{fails.}$$

Backtrack from c to f, then try g:

$$a \to b \to d \to f \to g \to e.$$

From e, both adjacent vertices b and g have already been used, while c remains unvisited. Hence, this branch also fails.

Backtrack further.

Successful branch

Choose e after b:

$$a \to b \to e \to g.$$

From g, choose d:

$$a \to b \to e \to g \to d.$$

Then choose f:

$$a \to b \to e \to g \to d \to f.$$

Next choose c:

$$a \to b \to e \to g \to d \to f \to c.$$

All seven vertices have now been visited exactly once. Since c is adjacent to the starting vertex a, return to a.

Therefore, a Hamiltonian circuit is

$$\boxed{a \to b \to e \to g \to d \to f \to c \to a}.$$

Verification

The required edges are:

$$(a,b), (b,e), (e,g), (g,d), (d,f), (f,c), (c,a),$$

and all of them exist in the graph.

Hence,

$$\boxed{a - b - e - g - d - f - c - a}$$

is a valid Hamiltonian circuit. Levitin's backtracking method similarly builds a Hamiltonian circuit one vertex at a time and backtracks whenever the partial path reaches a dead end. 212803334-Intro-to-Design-Analy...

3 (a) Describe best-first branch and bound approach with suitable example. [4]

(b) Give an example of best-case input for an assignment problem in context of branch and bound. [2]

(c) Solve the following knapsack problem using branch and bound algorithms. [8]

Item	Weight	Value
1	10	BDT 100
2	7	BDT 63
3	8	BDT 56
4	4	BDT 12

W = 16

3(a) Best-First Branch-and-Bound

Definition

Best-first Branch-and-Bound is an optimization technique that constructs a state-space tree and always expands the **most promising live node** first.

For every node, a bound on the best solution obtainable from that node is calculated:

- For a **minimization problem**, select the node with the smallest lower bound.
- For a **maximization problem**, select the node with the largest upper bound.

Live nodes are maintained in a **priority queue**. A node is pruned when its bound proves that it cannot improve the best solution found so far. Levitin applies Branch-and-Bound to the assignment, knapsack, and traveling-salesman problems in Section 12.2. 212803334-Intro-to-Design-Analy...

Main steps

1. Begin with the root representing no decisions.
2. Calculate its bound and insert it into a priority queue.
3. Remove the most promising live node.
4. Generate its children.
5. Discard infeasible children.
6. Prune any child whose bound cannot improve the current best solution.
7. Continue until no promising live node remains.

Suitable example

For the **assignment problem**, suppose the current live-node lower bounds are:

$$LB(A) = 20, \qquad LB(B) = 12, \qquad LB(C) = 17.$$

Because assignment is a minimization problem, best-first Branch-and-Bound expands node B first since

$$12 = \min(20, 12, 17).$$

If a complete assignment of cost 15 is later found, every live node with

$$LB \geq 15$$

is pruned.

3(b) Best-Case Input for an Assignment Problem

A best-case input is one in which the optimal assignment becomes obvious very early and all alternative branches can be pruned immediately.

For example:

$$C = \begin{bmatrix} 0 & 100 & 100 \\ 100 & 0 & 100 \\ 100 & 100 & 0 \end{bmatrix}.$$

The diagonal assignment is:

$$1 \to 1, \qquad 2 \to 2, \qquad 3 \to 3,$$

with total cost

$$0 + 0 + 0 = 0.$$

This is clearly optimal because no assignment can have a cost below zero. Every non-diagonal choice has cost at least 100, so all such branches are pruned.

Thus, this matrix represents a best-case input because Branch-and-Bound reaches the optimal solution quickly and explores only a small portion of the state-space tree.

3(c) Solve the 0–1 Knapsack Problem Using Branch-and-Bound

Given:

$$W = 16$$

Item	Weight w_i	Value v_i	v_i/w_i
1	10	100	10
2	7	63	9
3	8	56	7
4	4	12	3

The items are already arranged in decreasing order of value-to-weight ratio:

$$10 > 9 > 7 > 3.$$

For Branch-and-Bound, each node has:

$$(\text{level}, \text{weight}, \text{value}, \text{upper bound}).$$

The upper bound is calculated by filling the remaining capacity using the **fractional-knapsack method**. This gives an optimistic value, so a node can be pruned when

$$UB \leq \text{current best value}.$$

The book also uses decreasing value-to-weight ratios and a fractional upper bound for the knapsack Branch-and-Bound method. 212803334-Intro-to-Design-Analy...

Step 1: Root upper bound

At the root:

$$w = 0, \qquad v = 0.$$

Take item 1 completely:

$$w = 10, \qquad v = 100.$$

Remaining capacity:

$$16 - 10 = 6.$$

Take $\frac{6}{7}$ of item 2:

$$\frac{6}{7}(63) = 54.$$

Therefore,

$$UB(\text{root}) = 100 + 54 = 154.$$

State-Space Tree Exploration

We use:

- Left child: include the next item.
- Right child: exclude the next item.

Expand root

Node N_1: Include item 1

$$x_1 = 1$$

$$w = 10, \qquad v = 100.$$

Remaining capacity is 6. Fractionally take item 2:

$$UB(N_1) = 100 + \frac{6}{7}(63) = 154.$$

Since this is feasible,

$$\text{best} = 100.$$

Node N_2: Exclude item 1

$$x_1 = 0$$

Take items 2 and 3 completely:

$$w = 7 + 8 = 15,$$

$$v = 63 + 56 = 119.$$

Remaining capacity is 1. Take $\frac{1}{4}$ of item 4:

$$\frac{1}{4}(12) = 3.$$

Thus,

$$UB(N_2) = 119 + 3 = 122.$$

The priority queue contains:

$$N_1(UB = 154), \qquad N_2(UB = 122).$$

Expand N_1.

Expand $N_1 = (x_1 = 1)$

Include item 2

$$w = 10 + 7 = 17 > 16.$$

This node is infeasible and is discarded.

Exclude item 2

$$x_1 = 1, \qquad x_2 = 0.$$

Current:

$$w = 10, \qquad v = 100.$$

The remaining capacity is 6. Fractionally take item 3:

$$\frac{6}{8}(56) = 42.$$

Therefore,

$$UB = 100 + 42 = 142.$$

Call this node N_3.

Priority queue:

$$N_3(142), \qquad N_2(122).$$

Expand N_3.

Expand $N_3 = (1, 0)$

Include item 3

$$w = 10 + 8 = 18 > 16.$$

Therefore, this node is infeasible.

Exclude item 3

Current:

$$w = 10, \qquad v = 100.$$

Item 4 fits completely:

$$w = 10 + 4 = 14,$$

$$v = 100 + 12 = 112.$$

Hence,

$$UB = 112.$$

This gives the feasible solution

$$\{1, 4\}$$

with value

$$112.$$

Update:

$$\text{best} = 112.$$

Priority queue now contains:

$$N_2(122), \qquad N_4(112).$$

Expand N_2, since it has the greatest bound.

Expand $N_2 = (x_1 = 0)$

Include item 2

$$x_1 = 0, \qquad x_2 = 1.$$

Current:

$$w = 7, \qquad v = 63.$$

Item 3 fits completely:

$$w = 7 + 8 = 15,$$

$$v = 63 + 56 = 119.$$

Remaining capacity:

$$16 - 15 = 1.$$

Fractionally take item 4:

$$\frac{1}{4}(12) = 3.$$

Therefore,

$$UB = 119 + 3 = 122.$$

Call this node N_5.

Exclude item 2

Then only items 3 and 4 remain:

$$w = 8 + 4 = 12,$$

$$v = 56 + 12 = 68.$$

Therefore,

$$UB = 68.$$

Since

$$68 < 112,$$

this branch is pruned.

Priority queue:

$$N_5(122), \qquad N_4(112).$$

Expand N_5.

Expand $N_5 = (0, 1)$

Include item 3

$$x_1 = 0, \quad x_2 = 1, \quad x_3 = 1.$$

$$w = 7 + 8 = 15,$$

$$v = 63 + 56 = 119.$$

This is feasible, so update:

$$\text{best} = 119.$$

The remaining capacity is only 1. Fractionally using item 4 gives:

$$UB = 119 + \frac{1}{4}(12) = 122.$$

Exclude item 3

If item 3 is excluded, item 4 can be included:

$$w = 7 + 4 = 11,$$

$$v = 63 + 12 = 75.$$

Since

$$75 < 119,$$

this branch is pruned.

Final level

From the node containing items 2 and 3:

Include item 4

$$w = 7 + 8 + 4 = 19 > 16.$$

Infeasible.

Exclude item 4

$$w = 15, \qquad v = 119.$$

This is a complete feasible solution.

All remaining live nodes have bounds at most 112, which is less than the current best value 119. Therefore, they are pruned.

Node decisions	Weight	Value	Upper bound	Action
Root	0	0	154	Expand
$x_1 = 1$	10	100	154	Expand
$x_1 = 0$	0	0	122	Expand later
$x_1 = 1, x_2 = 1$	17	163	—	Infeasible
$x_1 = 1, x_2 = 0$	10	100	142	Expand
$(1, 0, 1)$	18	156	—	Infeasible
$(1, 0, 0)$	10	100	112	Later pruned
$(0, 1)$	7	63	122	Expand
$(0, 0)$	0	0	68	Pruned
$(0, 1, 1)$	15	119	122	Best feasible
$(0, 1, 0)$	7	63	75	Pruned
$(0, 1, 1, 1)$	19	131	—	Infeasible
$(0, 1, 1, 0)$	15	119	119	Optimal

Final Answer

The optimal items are:

$$\boxed{\text{Item 2 and Item 3}}.$$

Total weight:

$$7 + 8 = 15 \leq 16.$$

Maximum value:

$$63 + 56 = 119$$

The optimal items are:

$$\boxed{\text{Item 2 and Item 3}}.$$

Total weight:

$$7 + 8 = 15 \leq 16.$$

Maximum value:

$$63 + 56 = 119.$$

Therefore,

$$\boxed{\text{Maximum value} = \text{BDT } 119}$$

with solution vector

$$\boxed{(x_1, x_2, x_3, x_4) = (0, 1, 1, 0)}.$$

4(a) Write any string matching algorithm given a pattern P finds its occurrences in a text T. Analyze the complexity of the written algorithm in detail. [4]

(b) You are given a pattern P = "abbaab" and a text T = "abaaabbaabbbababa". Apply the KMP String matching algorithm to find the occurrences of P in T. Write the prefix function/array and simulate each step of your solution. [6]

(c) You are given a string S = "caabcba". Write an algorithm to remove the minimum number of characters to make S a palindrome. A palindrome is a string that reads the same from the beginning and the ending. Show the resultant string after applying your designed algorithm. [4]

4(a) Write Any String-Matching Algorithm and Analyze Its Complexity

For this question, the safest algorithm to write is the **naïve string-matching algorithm** because it is simple and easy to analyze.

Let:

- Text $T[1 \ldots n]$
- Pattern $P[1 \ldots m]$, where $m \leq n$

The goal is to find every shift s such that

$$T[s+1 \ldots s+m] = P[1 \ldots m].$$

Naïve String-Matching Algorithm

```
NAIVE-STRING-MATCHER(T, P)

n ← length(T)
m ← length(P)

for s ← 0 to n - m
    j ← 1

    while j ≤ m and T[s + j] = P[j]
        j ← j + 1

    if j = m + 1
        print "Pattern occurs with shift", s
```

Working principle

The pattern is aligned with every possible position of the text.

The possible shifts are

$$0, 1, 2, \ldots, n-m.$$

For every shift, the characters of P are compared with the corresponding characters of T.

If all m characters match, the shift is valid.

Complexity analysis

There are

$$n - m + 1$$

possible shifts.

At each shift, at most m character comparisons are performed.

Therefore, the maximum number of comparisons is

$$m(n - m + 1).$$

Hence, the worst-case running time is

$$\boxed{O((n - m + 1)m)}.$$

When $m \leq n$, this is commonly written as

$$\boxed{O(nm)}.$$

Worst case

The worst case occurs when many characters match before a mismatch occurs.

For example:

$$T = \texttt{AAAAAAAAAA}$$

$$P = \texttt{AAAAB}.$$

Almost the entire pattern is checked at every shift.

Thus,

$$\boxed{T_{\text{worst}}(n, m) = \Theta((n - m + 1)m)}.$$

Best case

If the first pattern character mismatches at every shift, only one comparison is performed per shift.

Therefore,

$$\boxed{T_{\text{best}}(n, m) = \Theta(n - m + 1) = \Theta(n)}.$$

Space complexity

The algorithm uses only a few variables.

Therefore,

$$\boxed{\text{Space complexity} = O(1)}.$$

4(b) Apply KMP

From the question:

$$P = \texttt{abbaab}$$

and the text appears to be

$$T = \texttt{abaaabbaabbbababa}.$$

The pattern has length

$$m = 6.$$

Step 1: Construct the prefix-function array

The prefix function $\pi[q]$ gives the length of the longest proper prefix of

$$P[1 \ldots q]$$

that is also a suffix of it.

Pattern:

Position q	1	2	3	4	5	6
$P[q]$	a	b	b	a	a	b

Now calculate each value.

$q = 1$

Substring:

$$\texttt{a}$$

No proper prefix exists.

$$\pi[1] = 0.$$

$q = 2$

Substring:

ab

Prefix **a** is not a suffix.

$$\pi[2] = 0.$$

$q = 3$

Substring:

abb

No proper prefix is also a suffix.

$$\pi[3] = 0.$$

$q = 4$

Substring:

abba

The longest matching prefix and suffix is

a.

Therefore,

$$\pi[4] = 1.$$

$q = 5$

Substring:

abbaa

The longest matching prefix and suffix is

a.

Therefore,

$$\pi[5] = 1.$$

$q = 6$

Substring:

$$\texttt{abbaab}$$

The longest matching prefix and suffix is

$$\texttt{ab}.$$

Therefore,

$$\pi[6] = 2.$$

Hence, the prefix array is

$$\boxed{\pi = [0, 0, 0, 1, 1, 2]}.$$

Step 2: KMP matching simulation

We use:

- i: current text position
- q: number of pattern characters currently matched

Text:

```
a b a a a b b a a b b b a b a b a
```

Pattern:

```
a b b a a b
```

Text position i	$T[i]$	Current q	Action	New q
1	a	0	$P[1] = a$, match	1
2	b	1	$P[2] = b$, match	2
3	a	2	$P[3] = b \neq a$, fallback to $\pi[1] = 0$; then match a	1
4	a	1	$P[2] = b \neq a$, fallback to 0; then match a	1
5	a	1	fallback to 0; match a	1
6	b	1	match	2
7	b	2	match	3
8	a	3	match	4
9	a	4	match	5
10	b	5	match	6

At $i = 10$,

$$q = m = 6.$$

Therefore, an occurrence has been found.

Starting position:

$$i - m + 1 = 10 - 6 + 1 = 5.$$

Thus, using 1-based indexing, the pattern occurs at position

$$\boxed{5}.$$

Its shift is

$$s = 5 - 1 = 4.$$

So,

$$\boxed{\text{valid shift } s = 4}.$$

After reporting the match,

$$q = \pi[6] = 2.$$

Continue searching:

i	$T[i]$	Action	New q
11	b	$P[3] = b$, match	3
12	b	mismatch with $P[4] = a$; fallback to $\pi[3] = 0$; mismatch again	0
13	a	match $P[1]$	1
14	b	match $P[2]$	2
15	a	mismatch; fallback to 0; match $P[1]$	1
16	b	match $P[2]$	2
17	a	mismatch; fallback to 0; match $P[1]$	1

No additional full match is found.

Final answer

Prefix-function array:

$$\boxed{[0, 0, 0, 1, 1, 2]}$$

Pattern occurrence:

$$\boxed{\text{Starting position } 5}$$

or equivalently,

$$\boxed{\text{valid shift } 4}.$$

Complexity

Prefix-function construction:

$$O(m).$$

Text scanning:

$$O(n).$$

Therefore, total KMP time is

$$\boxed{O(n+m)}.$$

Space complexity:

$$\boxed{O(m)}$$

4(c) Minimum Deletions to Make S a Palindrome

Given:

$$S = \mathbf{caabcba}.$$

We need to remove the minimum number of characters so that the remaining string is a palindrome.

Main idea

The minimum number of deletions is related to the **longest palindromic subsequence**, abbreviated LPS.

If

$$L = \text{length of the longest palindromic subsequence},$$

then

$$\boxed{\text{minimum deletions} = n - L}.$$

For

$$S = \mathbf{caabcba},$$

we have

$$n = 7.$$

A longest palindromic subsequence is

$$\mathbf{abcba},$$

whose length is

$$L = 5.$$

Therefore,

$$\text{minimum deletions} = 7 - 5 = 2.$$

The characters removed may be:

- the first **c**
- one of the repeated **a**'s near the beginning

Thus,

$$\mathbf{caabcba} \to \mathbf{abcba}.$$

The resulting string is a palindrome.

$$\boxed{\mathbf{abcba}}$$

Dynamic-programming algorithm

Let

$$dp[i][j]$$

denote the minimum number of deletions needed to make substring

$$S[i \ldots j]$$

a palindrome.

Recurrence

If

$$S[i] = S[j],$$

then the two end characters can remain:

$$dp[i][j] = dp[i+1][j-1].$$

Otherwise, delete either the left or right character:

$$dp[i][j] = 1 + \min\big(dp[i+1][j], dp[i][j-1]\big).$$

Base case

For a substring of length 1,

$$dp[i][i] = 0.$$

```
MINIMUM-PALINDROME-DELETIONS(S)

n ← length(S)

for i ← 0 to n - 1
    dp[i][i] ← 0

for length ← 2 to n
    for i ← 0 to n - length
        j ← i + length - 1

        if S[i] = S[j]
            if length = 2
                dp[i][j] ← 0
            else
                dp[i][j] ← dp[i + 1][j - 1]

        else
            dp[i][j] ← 1 + min(dp[i + 1][j],
                               dp[i][j - 1])

return dp[0][n - 1]
```

For this string,

$$\boxed{dp[0][6] = 2}.$$

Therefore, the minimum number of deletions is

$$\boxed{2}.$$

Complexity

There are $O(n^2)$ table entries, and each entry is calculated in constant time.

Therefore,

$$\boxed{\text{Time complexity} = O(n^2)}.$$

The DP table uses

$$\boxed{O(n^2)}$$

space.

Final answer

$$\boxed{\text{Minimum deletions} = 2}$$

and one resulting palindrome is

$$\boxed{\texttt{abcba}}.$$

5 (a) Explain the Marker's Algorithm. Apply the algorithm over the following pseudocode to extract the results. **[7]**

```
1. def func (a, b):
2.   c = a + b
3.   d = 2 * c - a
4.   e = 10
5.   f = 2 * d
6.   print(f)
```

(b) You are given the following system of congruences: **[7]**

- $x \equiv 2 \pmod 3$
- $x \equiv 3 \pmod 5$
- $x \equiv 2 \pmod 7$

Find the smallest positive integer that satisfies these congruences using the **Chinese Remainder Theorem (CRT)**.

5(a) Marker's Algorithm

Definition

Marker's Algorithm is a randomized online paging algorithm for a cache containing k pages.

Each page in the cache has a mark bit. Initially, all pages are unmarked.

For every requested page p:

1. If p is already in the cache, mark it.
2. If p is not in the cache and an unmarked page exists, randomly evict one unmarked page, load p, and mark p.
3. If p is not in the cache and all cached pages are marked, begin a **new phase**:
   - remove all marks;
   - randomly evict one page;
   - load and mark p.

A phase contains requests to at most k distinct pages. Once a page is requested during a phase, it is marked and cannot be evicted during that same phase.

Applying Marker's Algorithm to the pseudocode

Python Run

```python
def func(a, b):
    c = a + b
    d = 2 * c - a
    e = 10
    f = 2 * d
    print(f)
```

The question does not explicitly state the cache size. I will use the usual exam assumption:

$$\boxed{k = 3}$$

Treating each accessed variable as a page, the request sequence is

$$\boxed{a, b, c, c, a, d, e, d, f, f}.$$

Here, both reading and writing a variable count as an access.

Because Marker's Algorithm evicts randomly, different valid eviction choices may produce different cache contents. The following is one possible execution.

Simulation

Initially:

$$\text{Cache} = \varnothing$$

Request	Hit/Miss	Action	Cache after request	Marked pages
a	Miss	Load a	a	a
b	Miss	Load b	a, b	a, b
c	Miss	Load c	a, b, c	a, b, c
c	Hit	Already marked	a, b, c	a, b, c
a	Hit	Already marked	a, b, c	a, b, c
d	Miss	All marked; start new phase, clear marks, evict b, load d	a, c, d	d
e	Miss	Randomly evict unmarked c, load e	a, d, e	d, e
d	Hit	Mark remains	a, d, e	d, e
f	Miss	Evict unmarked a, load f	d, e, f	d, e, f
f	Hit	Already marked	d, e, f	d, e, f

The request sequence is divided into two phases:

$$\underbrace{a, b, c, c, a}_{\text{Phase 1}} \quad \Big| \quad \underbrace{d, e, d, f, f}_{\text{Phase 2}}.$$

Each phase contains at most $k = 3$ distinct pages.

Result

Page faults occur for:

$$a, b, c, d, e, f.$$

Therefore,

$$\boxed{\text{Total page faults} = 6}$$

and

$$\boxed{\text{Total cache hits} = 4}.$$

The exact evicted pages may differ because eviction is random, but the above is a valid simulation. Online caching algorithms organize requests into phases or epochs based on distinct cache blocks, which is central to their competitive analysis. Introduction_to_Algorithms_4th_...

5(b) Chinese Remainder Theorem

Given:

$$x \equiv 2 \pmod 3,$$

$$x \equiv 3 \pmod 5,$$

$$x \equiv 2 \pmod 7.$$

The moduli $3, 5, 7$ are pairwise relatively prime. Therefore, CRT guarantees a unique solution modulo

$$M = 3 \cdot 5 \cdot 7 = 105.$$

CLRS uses this same system as the standard CRT example and notes that all solutions are congruent to 23 modulo 105. Introduction_to_Algorithms_4th_...

Step 1: Calculate M_i

$$M_1 = \frac{105}{3} = 35,$$

$$M_2 = \frac{105}{5} = 21,$$

$$M_3 = \frac{105}{7} = 15.$$

Step 2: Find multiplicative inverses

Find y_1 such that

$$35y_1 \equiv 1 \pmod{3}.$$

Since

$$35 \equiv 2 \pmod{3}$$

and

$$2 \cdot 2 \equiv 1 \pmod{3},$$

we get

$$y_1 = 2.$$

Next,

$$21y_2 \equiv 1 \pmod{5}.$$

Since

$$21 \equiv 1 \pmod{5},$$

$$y_2 = 1.$$

Finally,

$$15y_3 \equiv 1 \pmod{7}.$$

Since

$$15 \equiv 1 \pmod{7},$$

$$y_3 = 1.$$

$y_3 = 1$.

Step 3: Apply the CRT formula

$$x \equiv a_1 M_1 y_1 + a_2 M_2 y_2 + a_3 M_3 y_3 \pmod{M}.$$

Therefore,

$$\begin{aligned} x &\equiv 2(35)(2) + 3(21)(1) + 2(15)(1) \pmod{105} \\ &\equiv 140 + 63 + 30 \pmod{105} \\ &\equiv 233 \pmod{105}. \end{aligned}$$

Now,

$$233 = 2(105) + 23.$$

Thus,

$$\boxed{x \equiv 23 \pmod{105}}.$$

All integer solutions are

$$\boxed{x = 23 + 105k, \qquad k \in \mathbb{Z}}.$$

Hence, the smallest positive solution is

$$\boxed{x = 23}.$$

Verification

$$23 \bmod 3 = 2,$$

$$23 \bmod 5 = 3,$$

$$23 \bmod 7 = 2.$$

Therefore, the answer is correct.

6 (a) State the decision version of independent set and vertex cover problem. **[2]**

(b) **To compare these two problems mentioned in 6 (a), we need to establish a relationship** between these two problems which enables us to transform the input instance of one problem into the input instance of another problem. State the relationship in the form of lemma and prove it. **[6]**

(c) Give a 2-approximation Vertex cover algorithm. Prove that the approximation ratio of the algorithm is 2. **[6]**

6(a) Decision Versions

Let $G = (V, E)$ be an undirected graph and let k be a positive integer.

Independent Set — Decision Version

An **independent set** is a subset $S \subseteq V$ such that no two vertices in S are adjacent.

The decision problem is:

> Does G contain an independent set S such that
>
> $$|S| \geq k?$$

Vertex Cover — Decision Version

A **vertex cover** is a subset $C \subseteq V$ such that every edge in E has at least one endpoint in C.

The decision problem is:

> Does G contain a vertex cover C such that
>
> $$|C| \leq k?$$

These are the standard decision formulations used in the NP-completeness material. P vs NP

6(b) Relationship Between Independent Set and Vertex Cover

Lemma

For any undirected graph $G = (V, E)$, a subset $S \subseteq V$ is an independent set if and only if its complement

$$V - S$$

is a vertex cover.

Equivalently,

$$\boxed{S \text{ is an independent set} \iff V - S \text{ is a vertex cover}}$$

Therefore,

$$\boxed{G \text{ has an independent set of size } k \iff G \text{ has a vertex cover of size } |V| - k}$$

This gives the polynomial-time reduction

$$\boxed{\text{INDEPENDENT-SET} \leq_p \text{VERTEX-COVER}}.$$

Proof

Forward direction

Assume S is an independent set.

Since no two vertices of S are adjacent, there cannot be an edge whose both endpoints lie inside S.

Therefore, every edge of G must have at least one endpoint outside S, that is, in

$$V - S.$$

Hence, $V - S$ covers every edge, so it is a vertex cover.

Thus,

$$S \text{ independent} \implies V - S \text{ vertex cover}.$$

Reverse direction

Assume $V - S$ is a vertex cover.

Then every edge has at least one endpoint in $V - S$. Therefore, no edge can have both endpoints inside S.

Hence, no two vertices in S are adjacent, and so S is an independent set.

Thus,

$$V - S \text{ vertex cover} \implies S \text{ independent.}$$

Therefore,

$$\boxed{S \text{ is independent} \iff V - S \text{ is a vertex cover}}.$$

Transformation of instances

Given an instance

$$(G, k)$$

of Independent Set, construct the Vertex Cover instance

$$\boxed{(G, |V| - k)}.$$

Then,

$$G \text{ has an independent set of size at least } k$$

if and only if

$$G \text{ has a vertex cover of size at most } |V| - k.$$

The transformation only computes $|V| - k$, so it takes polynomial time. P vs NP

6(c) A 2-Approximation Algorithm for Vertex Cover

Algorithm

```
APPROX-VERTEX-COVER(G)

1. C ← ∅
2. E' ← E

3. while E' ≠ ∅
4.     choose any edge (u, v) from E'
5.     C ← C ∪ {u, v}
6.     remove from E' every edge incident on u or v

7. return C
```

The algorithm repeatedly chooses an uncovered edge, includes both endpoints in the cover, and removes all edges covered by those endpoints. This is the algorithm given in the approximation slides and CLRS.

23_approx Introduction_to_Algorithms_4th_...

Correctness

When the algorithm terminates,

$$E' = \varnothing.$$

Every original edge has therefore been removed because it was incident on some selected vertex in C.

Hence, every edge has at least one endpoint in C, so C is a valid vertex cover.

Proof of Approximation Ratio 2

Let:

- C be the vertex cover returned by the algorithm.
- C^* be an optimal minimum vertex cover.
- A be the set of edges selected in line 4.

No two edges in A share an endpoint. Once an edge (u, v) is selected, every edge incident on u or v is removed.

Therefore, A is a matching.

To cover every edge in A, any vertex cover, including the optimal cover C^*, must contain at least one endpoint of each edge in A.

Since the edges in A are disjoint, one vertex cannot cover two different edges of A. Therefore,

$$\boxed{|C^*| \geq |A|}.$$

For every selected edge, the algorithm adds both endpoints to C. Hence,

$$\boxed{|C| = 2|A|}.$$

Combining the two results,

$$|C| = 2|A| \leq 2|C^*|.$$

Thus,

$$\frac{|C|}{|C^*|} \leq 2.$$

Therefore,

$$\boxed{\text{APPROX-VERTEX-COVER is a 2-approximation algorithm.}}$$

This is the standard CLRS proof: the selected edges form a maximal matching, their number lower-bounds the optimum cover, and the algorithm returns exactly two endpoints per selected edge.

Introduction_to_Algorithms_4th_...

Time complexity

Using adjacency lists, each vertex and edge is processed only a constant number of times. Therefore,

$$\boxed{O(|V| + |E|)}.$$

Final exam conclusion

7 (a) SKI rental problem is defined as follows. [4]

"You are going skiing for an unknown number of days. Assume that renting skis costs 500 Taka per day and buying costs 1500 Taka. Every day you have to decide whether to continue renting skis for one more day or buy a pair of skis"

Mention the features of online algorithm. Define the competitive ratio using three strategy of SKI rental problem defined above.

(b) Consider the following linear list search problem

"Given a set of items in a list where the cost of accessing an item is proportional to the distance from the head of the list i.e., a linked list, and a request sequence of accesses, the problem is to come up with a strategy of reordering the list so that the total cost of access is minimized"

i) Consider a case in which move the accessed item two positions forward and request sequence is only for the last item of the list. Calculate the competitive ration. [4]

ii) Prove that no algorithm has Competitive ratio better than 2 for linear list search problem. [6]

7(a) Ski-Rental Problem

An **online algorithm** receives input gradually and must make each decision without knowing future requests. Its performance is evaluated through **competitive analysis**, by comparing its cost with an optimal offline algorithm OPT, which knows the entire future input. Introduction_to_Algorithms_4th_...

Features of an online algorithm

1. The complete input is not known beforehand.
2. Input arrives one request at a time.
3. A decision must be made immediately.
4. Earlier decisions usually cannot be reversed without cost.
5. Performance is compared with an optimal offline algorithm.
6. The analysis considers the worst possible input sequence.

Competitive ratio

For a minimization problem, the competitive ratio of an online algorithm A is

$$CR(A) = \max_{\sigma} \frac{A(\sigma)}{OPT(\sigma)}.$$

An algorithm is c-competitive if

$$A(\sigma) \leq c \cdot OPT(\sigma)$$

for every input sequence σ.

Ski-rental setting

- Rent cost per day $= 500$ Taka
- Buying cost $= 1500$ Taka
- Total skiing days $= d$, which is unknown

The offline optimum is

$$OPT(d) = \min(500d, 1500).$$

Thus,

$$OPT(d) = \begin{cases} 500d, & d < 3, \\ 1500, & d \geq 3. \end{cases}$$

The slides consider three online strategies. Online Algorithm

Strategy 1: Always rent

For d days,

$$A_1(d) = 500d.$$

For $d \geq 3$,

$$OPT(d) = 1500.$$

Therefore,

$$\frac{A_1(d)}{OPT(d)} = \frac{500d}{1500} = \frac{d}{3}.$$

As $d \to \infty$,

$$\frac{d}{3} \to \infty.$$

Hence,

$$\boxed{CR(A_1) = \infty}.$$

Therefore, always renting is not competitive.

Strategy 2: Buy on the first day

The online cost is always

$$A_2(d) = 1500.$$

The worst case occurs when $d = 1$, because

$$OPT(1) = 500.$$

Therefore,

$$CR(A_2) = \frac{1500}{500} = 3.$$

Hence,

$$\boxed{CR(A_2) = 3}.$$

Strategy 3: Rent for two days and buy on the third day

Case 1: $d < 3$

The algorithm only rents:

$$A_3(d) = 500d.$$

The offline algorithm also rents:

$$OPT(d) = 500d.$$

Therefore,

$$\frac{A_3(d)}{OPT(d)} = 1.$$

Case 2: $d \geq 3$

The online algorithm rents for two days and then buys:

$$A_3(d) = 2(500) + 1500 = 2500.$$

The offline optimum buys for

$$OPT(d) = 1500.$$

Therefore,

$$\frac{A_3(d)}{OPT(d)} = \frac{2500}{1500} = \frac{5}{3}.$$

Hence,

$$\boxed{CR(A_3) = \frac{5}{3}}.$$

Comparison

Strategy	Competitive ratio
Always rent	∞
Buy on day 1	3
Rent for 2 days, buy on day 3	$\frac{5}{3}$

Therefore, among the three strategies,

$$\boxed{\text{Rent for two days and buy on the third day is the best.}}$$

7(b)(i) Moving the Accessed Item Forward by Two Positions

Suppose the initial list is

$$1, 2, 3, \ldots, n,$$

and the request sequence consists of n requests for the last item:

$$\sigma = n, n, n, \ldots, n.$$

After every access, the requested item is moved two positions toward the front.

Its successive access costs are approximately

$$n,\ n-2,\ n-4, \ldots$$

until it reaches the front. The remaining accesses cost 1.

Therefore, the online cost is

$$A(\sigma) = \Theta(n^2).$$

More precisely, for even $n = 2r$,

$$\begin{aligned} A(\sigma) &= 2r + (2r-2) + \cdots + 2 + r \\ &= r(r+1) + r \\ &= r^2 + 2r \\ &= \frac{n^2}{4} + n. \end{aligned}$$

The optimal offline algorithm accesses item n once at cost n, moves it immediately to the front, and pays 1 for each remaining request:

$$OPT(\sigma) = n + (n-1) = 2n - 1 < 2n.$$

Thus,

$$\begin{aligned} \frac{A(\sigma)}{OPT(\sigma)} &> \frac{\frac{n^2}{4}}{2n} \\ &= \frac{n}{8}. \end{aligned}$$

As $n \to \infty$,

$$\frac{n}{8} \to \infty.$$

Therefore,

$$\boxed{CR(A) = \infty}.$$

Hence, moving an accessed item forward by only two positions is not constant-competitive. Repeated requests for a distant item cause quadratic online cost, while OPT has only linear cost. Online Algorithm

7(b)(ii) Lower Bound for Linear List Search

Theorem

No deterministic online algorithm for linear list search can have a competitive ratio better than

$$\boxed{2 - \frac{2}{n+1}}.$$

Therefore, as $n \to \infty$, no deterministic algorithm has an asymptotic competitive ratio better than 2.

Proof

Consider any deterministic online algorithm A on a list of n items.

An adversary always requests the item currently at the last position of A's list. Thus, every request costs n.

For a sequence of m requests,

$$\boxed{A(\sigma) = mn}.$$

Now consider all $n!$ fixed permutations of the list:

$$\pi_1, \pi_2, \ldots, \pi_t, \qquad t = n!.$$

Let $C_{i,j}$ be the cost of serving the j-th request using permutation π_i. Then

$$O(\pi_i) = C_{i,1} + C_{i,2} + \cdots + C_{i,m} = \sum_{j=1}^{m} C_{i,j}.$$

The minimum cost among the permutations is no greater than their average:

$$\min_i O(\pi_i) \leq \frac{O(\pi_1) + O(\pi_2) + \cdots + O(\pi_t)}{n!}.$$

Substituting $O(\pi_i)$ and changing the order of summation,

$$\begin{aligned} \text{Average} &= \frac{\sum_{j=1}^{m} C_{1,j} + \sum_{j=1}^{m} C_{2,j} + \cdots + \sum_{j=1}^{m} C_{t,j}}{n!} \\ &= \frac{\sum_{i=1}^{t} C_{i,1} + \sum_{i=1}^{t} C_{i,2} + \cdots + \sum_{i=1}^{t} C_{i,m}}{n!}. \end{aligned}$$

$n!$

For any requested item x_j, across all $n!$ permutations, it occurs in each position $1, 2, \ldots, n$ exactly $(n-1)!$ times. Hence,

$$\begin{aligned}\sum_{i=1}^{t} C_{i,j} &= (n-1)!(1+2+\cdots+n) \\ &= (n-1)!\frac{n(n+1)}{2} \\ &= \frac{n!(n+1)}{2}.\end{aligned}$$

Therefore, for m requests,

$$\begin{aligned}\text{Average} &= \frac{m \cdot \frac{n!(n+1)}{2}}{n!} \\ &= \frac{m(n+1)}{2}.\end{aligned}$$

Thus, at least one fixed permutation has cost at most this average. Since OPT is no worse than that permutation,

$$OPT(\sigma) \leq \frac{m(n+1)}{2}.$$

Therefore,

$$\begin{aligned}\frac{A(\sigma)}{OPT(\sigma)} &\geq \frac{mn}{\frac{m(n+1)}{2}} \\ &= \frac{2n}{n+1} \\ &= 2 - \frac{2}{n+1}.\end{aligned}$$

Hence,

$$\boxed{CR(A) \geq 2 - \frac{2}{n+1}}.$$

Therefore, no deterministic online algorithm for linear list search can have a better competitive ratio than

$$\boxed{2 - \frac{2}{n+1}}.$$

As

$$n \to \infty,$$

$$2 - \frac{2}{n+1} \to 2.$$

Since Move-to-Front is 2-competitive, it is asymptotically optimal. Online Algorithm